Haystack vs LightRAG: Benchmark Report

Axel Tang
University of Toronto
Fujifilm
axel.tang@mail.utoronto.ca
axel.tang.mn@fujifilm.com

Abstract

This report compares two ways of answering TriviaQA questions: a simple Haystack setup that finds the right text with BM25, and a LightRAG setup that also tries to build and use a graph. We ran 250 questions in the final benchmark and the key numbers are shown below.

Summary (n=250): Haystack got 100.0% EM and 100.0% F1. LightRAG got 99.6% EM and 99.8% F1. Both found the right evidence every time, but LightRAG was much slower and still had some graph-building problems on larger runs.

Attestation of Teamwork

Personal Usage

1 Description

This project checks which system answers TriviaQA questions better: Haystack, which mostly just finds the right text and copies the answer, and LightRAG, which tries to do that plus extra graph work. The goal is to see how accurate and how fast they are.

2 Problem Statement

We compare two approaches to retrieval-augmented QA:

- **Haystack (baseline)**: find the most relevant chunks with BM25, then pull the answer straight from the text.
- **LightRAG**: use an LLM to pick out entities and relations, build a graph, and then answer from that graph plus the text.

The big problem we saw with LightRAG is simple: if the graph step fails, the graph can end up empty. When that happens, the answers get much worse or drop to zero. We added backups, like BM25 grounding and fallback graph files, so the benchmark still runs.

3 Three-Layer View

Retriever Haystack uses BM25; LightRAG uses graph search with BM25 backup.

Knowledge artifact Haystack works with plain text chunks; LightRAG also keeps graph files.

Answer composer Haystack copies the answer from the best chunk; LightRAG asks the LLM to reason over the graph and text.

4 Methodology

We test both pipelines on TriviaQA `rc`, which already contains the supporting text. For each question, we check Exact Match (EM), token F1, recall@5, and grounded EM.

The main 250-question run is saved at `results/benchmark250`q`20260506011939.json`. We also ran smaller tests and a hard-mode test that hides the obvious answer text.

Table 1: Hyperparameters and setup

Parameter	Value
Dataset	TriviaQA (rc) – 250 questions
Retriever	BM25 (baseline); LightRAG graph + BM25 fallback
Top-k	5 (for reported retrieved chunks)
Timeouts	per-component timeouts used in config (embed/gen/insert)
Notes	LightRAG includes parser retry, JSON extraction enforcement, and raw-response logging

5 Results (n=250)

Table 2: Summary metrics for Haystack vs LightRAG (250 questions)

Method	EM (%)	F1 (%)	recall@5	grounded EM	avg latency (s)	qps
Haystack	100.0	100.0	1.00	1.00	0.00051	1978.14
LightRAG	99.6	99.8	1.00	1.00	0.00377	265.13

Per-question results and retrieved chunks are in the JSON file. A few simple takeaways:

- Haystack is very fast and works well when the answer is already in the text.
- LightRAG can get almost the same accuracy, but it is slower because it does more work.
- LightRAG still has trouble building the graph cleanly on bigger runs.

6 Implementation Notes and Fixes

During development we implemented multiple resilience improvements in the LightRAG pipeline:

- We forced the extraction output to be machine-readable JSON and retried parsing a few times.
- We logged raw LLM prompts and replies so we could see what went wrong.
- We added BM25 fallback so LightRAG still has useful retrieved chunks even if the graph is empty.
- We also wrote fallback entity and relation files when graph insert failed.

These changes fixed the earlier all-zero runs and made the results usable again, but the graph insert path still needs better batching and timeout settings.

7 Discussion

The main lesson is easy to state: if the corpus already contains the answer, a simple BM25 system like Haystack is usually the safest and fastest choice. LightRAG can be just as accurate when everything works, but it is more complicated and slower. If the graph or extraction step breaks, the scores can fall apart unless we add backups.

7.1 Why hard-mode LLM scores can drop to zero

In hard mode, we removed the easiest clue from the text: the `Canonical answer:` label. We also turned off the shortcut that copied the answer directly from the chunk. So now the LLM has to guess from weaker context, but the benchmark still wants the exact TriviaQA answer. That is why EM and F1 can drop to zero even when the right chunk is still found.

So the hard-mode run is not the same kind of test as the earlier runs. The earlier near-perfect scores were helped a lot by answer leakage in the corpus format. Hard mode is a stricter test of real answer generation from weaker evidence.

Table 3: Recent benchmark runs and metric outcomes

Run	Mode	N	EM (%)	F1 (%)	recall@5	grounded EM
20260506_013738	deterministic / fallback LLM mode	250	100.0	100.0	1.00	1.00
20260506_014125	LLM vs LLM	50	100.0	100.0	1.00	1.00
20260506_014358	hard-mode LLM vs LLM	50	0.0	0.0	1.00	0.0

The table shows the pattern clearly: when the answer text is visible, both pipelines look perfect; when we hide that clue, the exact-match score drops to zero because the remaining evidence is too weak for the current prompts.

8 Conclusions and Recommendations

- If you want speed and simple answers, Haystack is the better choice.
- If you want graph-based reasoning, LightRAG can help, but it needs stronger extraction and better graph insert handling.
- After we fix batching and timeouts, we should run the 250-question benchmark again.

Artifacts

- Results JSON: `results/benchmark`$_2$`50`q_2`0260506`$_0$`11939`$.json$
- `LightRAG logs:` `lightrag`$_p ipeline/lightrag_r aw_r esponses.log$
- `Fallback entity files (if any):` `lightrag`$_p ipeline/fallback_e ntities.json$

Appendix: Reproducible commands

Run the final benchmark (example):

```
source .venv/bin/activate && \
env SKIP_BOOTSTRAP_CHECK=1 python3 scripts/run_benchmark.py 250
```

Inspect the saved results:

```
python3 scripts/report.py results/benchmark_250q_20260506_011939.json
```